

# ARTHAYOG DORMITORY

Complete ERP & Production Application Specification  
Updated Functional, Security & Deployment Requirements

**Purpose:** Build a real, secure, production-ready dormitory ERP for a 16-bed property. Support Owner/Admin operations, Staff phone bookings and optional Customer self-booking.

**Core rule:** Every operational feature must work through Frontend → Backend API → Persistent Database. No fake production data, fake payments or frontend-only authorization.

# OVERVIEW

Arthayog Dormitory is a real-world dormitory ERP for a small-town property. It manages reservations, payments, guests, beds, check-in/check-out, cleaning, inventory, maintenance, staff tasks, reports and audit history.

- 16 total beds: Floor 1 = 5, Floor 2 = 6, Floor 3 = 5.
- Typical stays are 1 day, with some 3–4 day stays.
- Many bookings are for weddings/events and are handled by phone.
- Staff can create phone bookings; customers can optionally self-book.
- Payment must be verified before a booking is confirmed.
- The system is mobile-first.

# INTRODUCTION

The system replaces fragmented manual processes with one secure platform connecting booking, payment and daily dormitory operations.

- Owner receives complete control and reporting.
- Staff receives fast tools for daily operations.
- Customers can optionally book themselves.
- The database is the single source of truth for availability, bookings, payments and inventory.

# PROBLEM STATEMENT

The dormitory needs dependable management of reservations, payments, bed availability, cleaning, supplies and maintenance while protecting real customer data.

- Prevent double-bookings.
- Release beds when unpaid provisional bookings expire.
- Avoid random or hardcoded dashboard information.
- Provide mobile workflows for staff.
- Protect personal and payment-related information.

# GOALS

Create a practical ERP that improves operations while providing strong security and reliable persistent records.

- Manage all 16 beds with date-aware availability.
- Support Owner/Admin, Staff/Employee and Customer/Guest roles.
- Support phone and optional online bookings.
- Require verified payment before confirmation.
- Track check-in/out, cleaning, maintenance, tasks and inventory.
- Provide real reports and owner-only audit logs.
- Protect credentials, personal data, payment secrets and database access.

# SOLUTION

Use a mobile-first full-stack application where the frontend communicates with secure backend APIs and the backend validates permissions, business rules and database operations.

- React responsive frontend.
- FastAPI/backend API layer.
- PostgreSQL-compatible persistent database such as PostgreSQL/Supabase.
- Secure email/password authentication with Argon2id.
- Backend-enforced role authorization.
- Real Indian payment gateway with server-side verification/webhooks.

# STRATEGY

Build around connected real-world workflows rather than isolated screens.

- Booking, payment, bed status, check-in, checkout, cleaning, inventory and reports share persistent records.
- Use backend-controlled business rules.
- Use database constraints/transactions where needed.
- Dashboard numbers must come from the database.
- Test every major module before moving forward.

# USER ROLES & ACCESS CONTROL

Three roles are required and authorization must be enforced by the backend.

- Owner/Admin: complete system control.
- Staff/Employee: daily operations and permitted records.
- Customer/Guest: only their own account, bookings and payment information.
- Customers cannot obtain admin access by modifying frontend code or API requests.
- Only Owner/Admin can create or invite staff/admin accounts.

# AUTHENTICATION & SECURITY

Because the system may contain real customer and payment information, security is mandatory.

- Argon2id password hashing; never plaintext passwords.
- Email verification and secure password reset.
- Secure session/JWT strategy.
- Secrets and payment/database credentials only in backend environment variables.
- Never commit secrets to source control.
- HTTPS in production.
- Backend input validation and secure error handling.
- Parameterized queries/ORM methods to reduce SQL injection risk.
- Rate limiting/brute-force protection where appropriate.
- Never expose passwords, tokens, API keys or payment secrets in logs, URLs or frontend code.

# CUSTOMER PRIVACY

All customer, guest, booking and staff information must be treated as confidential and protected using least-privilege access.

- Customers can access only their own information.
- Staff access only information required for their work.
- Do not expose unnecessary personal data in API responses.
- Do not use real customer data during development/testing.
- Use test accounts and sandbox payment data until production verification.
- Apply appropriate retention/deletion controls.

# BED & PROPERTY MANAGEMENT

The property contains 16 beds distributed across three floors. Availability must be date-aware.

- Floor 1: 5 beds.
- Floor 2: 6 beds.
- Floor 3: 5 beds.
- Track available, reserved, occupied, cleaning and maintenance states.
- Prevent overlapping bookings.
- After checkout and cleaning, a bed returns to ready/available.
- Maintenance can remove a bed from availability.

# BOOKING ENGINE

Support staff phone reservations and optional customer self-booking. Unpaid provisional bookings must expire and release inventory.

- Statuses: PENDING\_PAYMENT, CONFIRMED, CHECKED\_IN, CHECKED\_OUT, CANCELLED, EXPIRED.
- Date-aware availability prevents double-booking.
- Authorized staff can create phone bookings.
- Customers can create online bookings.
- Expired unpaid bookings release beds.
- Confirmation occurs only after verified payment.
- Persist booking and cancellation history.

# PAYMENT SYSTEM

Use a genuine Indian payment gateway supporting appropriate UPI/payment methods. Do not create fake Google Pay or fake success functionality.

- Statuses: PENDING, SUCCESS, FAILED and REFUNDED where supported.
- Payment secrets remain backend-only.
- Backend verifies gateway status/webhooks.
- Never trust browser-only payment success.
- Prevent duplicate payment processing.
- Use sandbox/test payments during development.
- Use live credentials only after full verification.

# OWNER / ADMIN PANEL

The owner dashboard must use real database information.

- Total beds: 16.
- Available/reserved/occupied beds.
- Today's check-ins/check-outs.
- Pending payments.
- Actual successful-payment revenue.
- Low/out-of-stock supplies.
- Pending staff tasks.
- Staff management, reports and owner-only audit logs.
- No random, fake or hardcoded statistics.

# STAFF / EMPLOYEE PANEL

Designed for fast mobile daily operations, especially phone bookings.

- Real availability.
- Phone booking.
- Payment request/status.
- Check-in and checkout.
- Cleaning and bed-ready workflow.
- Supply usage.
- Maintenance.
- Assigned staff tasks.

# CUSTOMER PORTAL

Optional self-service booking for customers.

- Create account and verify email.
- Secure login.
- Check date-based availability.
- Enter booking details.
- Pay through real gateway.
- Receive confirmation only after verified payment.
- View own current and past bookings only.

# CHECK-IN, CHECKOUT & CLEANING

Bed state must reflect actual operational readiness.

- Occupied → Checked Out.
- Checked Out → Cleaning Required.
- Cleaning Required → Cleaning In Progress.
- Cleaning In Progress → Ready.
- Only ready/available beds can be offered, subject to dates and maintenance.

# INVENTORY & SUPPLIES

Track operational supplies using persistent records.

- Bedsheets, towels, blankets, soap, toilet paper, drinking water and cleaning supplies.
- Purchased, used and remaining quantities.
- Purchase and usage history.
- Low-stock/out-of-stock alerts.
- All changes persist in the database.

# MAINTENANCE

Track property problems and prevent affected resources from being incorrectly offered.

- Bed/equipment, bathroom, electrical, plumbing and other categories.
- Create, assign, prioritize, update and resolve.
- Track maintenance history.
- Unavailable beds/equipment must not be offered.

# STAFF TASKS

Provide assignable daily operational tasks.

- Cleaning.
- Bed preparation.
- Bedsheet replacement.
- Bathroom checks.
- Water refill.
- Checkout inspection.
- Maintenance.
- Assigned staff, status, timestamps and completion.

# REPORTS & HISTORY

Reports must be generated from real persistent records.

- Daily/weekly/monthly bookings.
- Occupancy and bed usage.
- Revenue and payment status.
- Cancelled/expired bookings.
- Guest history.
- Floor/bed usage.
- Supply purchases and consumption.
- Staff task completion.
- Maintenance history.

# AUDIT LOGGING

Keep traceable owner-only records of important actions.

- User, role, action, affected record and timestamp.
- Before/after values where appropriate.
- Owner/Admin-only access.
- Never log passwords, payment secrets or unnecessary sensitive data.
- Never generate fake/random audit entries.

# NOTIFICATIONS

Provide secure notifications for account and operational events.

- Email verification.
- Password reset.
- Booking/payment confirmation where configured.
- Payment failure/pending notices.
- Low-stock alerts.
- Keep email credentials in backend environment variables.

# MOBILE-FIRST UX

The system will mainly be used on phones.

- Touch-friendly controls.
- Responsive dashboards and forms.
- No horizontal scrolling.
- Fast staff operations.
- Clear loading, success, empty and error states.

# ARCHITECTURE

The intended flow is: user interacts with React frontend → frontend sends request to FastAPI/backend → backend authenticates and authorizes → business rules execute → database/payment service is accessed → result returns to frontend.

- Frontend: React.
- Backend: FastAPI/API.
- Database: PostgreSQL-compatible persistent storage.
- Authentication: Argon2id plus secure session/JWT approach.
- Payment: external gateway with backend verification/webhooks.
- Deployment: HTTPS with protected environment secrets.

# NO FAKE PRODUCTION FUNCTIONALITY

The final system must behave like a real application, not a UI mockup.

- No fake payment success.
- No random users.
- No random audit logs.
- No hardcoded revenue/occupancy.
- No simulated availability.
- No placeholder buttons.
- No frontend-only authorization.
- No localStorage as production source of truth for bookings/payments.

# SECURITY & RECOVERY

Production safety includes prevention, access control and recovery.

- Least-privilege database access.
- Protected backend/database credentials.
- HTTPS.
- Regular database backups.
- Test backup restoration.
- Monitoring/error logging without secrets.
- Review permissions before production.
- Test unauthorized access to protected resources.

# IMPLEMENTATION STRATEGY

Implement in controlled phases and test after each phase.

- Inspect existing project and preserve useful code.
- Configure database and environment secrets.
- Implement authentication and authorization.
- Implement beds and date-aware availability.
- Implement bookings.
- Implement payment sandbox and verification.
- Implement owner, staff and customer panels.
- Implement cleaning, inventory, maintenance and tasks.
- Implement reports, notifications and audit logs.
- Apply branding and mobile UX.
- Perform security, end-to-end and recovery testing.
- Deploy only after acceptance criteria pass.

# TESTING & ACCEPTANCE

Do not declare production-ready until critical workflows and security boundaries pass.

- Owner, staff and customer workflows work end-to-end.
- Unauthorized role access is rejected by backend.
- Passwords are securely hashed.
- Email verification/reset works.
- Double-booking is prevented.
- Expired unpaid bookings release inventory.
- Payment success is server-verified.
- Failed payments do not confirm bookings.
- Check-in/out/cleaning states work.
- Inventory and reports reflect database values.
- Audit logs contain real actions only.
- Mobile layouts work.
- Production secrets are not exposed.
- Backup and restore procedure is tested.

## FUTURE SCOPE

After the core system is stable, additional capabilities can be added without disrupting core workflows.

- Improved online booking.
- WhatsApp/SMS notifications through a suitable provider.
- Advanced revenue and occupancy analytics.
- Digital receipts/invoices.
- Multi-property support.
- Staff scheduling integrations.
- More detailed owner analytics.

# CONCLUSION

Arthayog Dormitory is specified as a practical, secure and production-oriented ERP rather than a simple booking UI. It connects 16-bed inventory, phone and online bookings, verified payments, check-in/out, cleaning, supplies, maintenance, staff operations, reporting and audit history through a persistent backend.

- Security, privacy and backend authorization are mandatory.
- Use sandbox/test data until security and payment workflows are verified.
- Every important operation must connect to the real backend and database.
- Production deployment follows complete end-to-end testing.

# FINAL ANTIGRAVITY IMPLEMENTATION INSTRUCTION

READ THIS ENTIRE SPECIFICATION BEFORE IMPLEMENTATION.

Build Arthayog Dormitory as a genuine production-ready dormitory ERP. Inspect the existing project first and preserve useful working code. Implement every requirement in this document. Every operational feature must work through Frontend → Backend API → Persistent Database.

Implement secure authentication, Argon2id password hashing, email verification, password reset, backend-enforced Owner/Admin, Staff/Employee and Customer/Guest authorization, date-aware 16-bed booking, phone booking, optional customer self-booking, real payment-gateway integration with backend verification, check-in/out, cleaning, inventory, maintenance, staff tasks, reports, notifications and owner-only audit logs.

Treat customer, booking, payment and staff information as confidential. Never expose passwords, tokens, API keys, database credentials or payment secrets. Never use real customer data for development. Use sandbox/test payment credentials until all workflows and security checks pass. Do not use fake/random dashboard data, fake users, fake audit logs, fake payment success, hardcoded availability or placeholder buttons.

Prevent double bookings, expire unpaid provisional bookings, enforce role permissions on the backend, validate inputs, protect database access, use HTTPS in production, keep secrets in backend environment variables, apply appropriate rate limiting and secure error handling, and maintain reliable database backups and recovery procedures.

After each major phase, run and test the application, verify frontend/backend/database behavior, check authorization and fix errors before continuing. At the end, perform complete end-to-end testing for Owner/Admin, Staff and Customer, including unauthorized access, failed/duplicate payments, duplicate booking attempts, expired bookings, check-in/out, cleaning, inventory, maintenance and reports.

Do not declare the application production-ready until all acceptance requirements in this specification pass.